FR1FR2TTUG

FR1 FR2 Test Tool User Guide

Rev 5.1 — NOV 2025

User guide

CONFIDENTIAL

Document information

Information	Content
Keywords	FR1FR2TTUG, FR1, FR2, FR1 FR2 Test Tool
Abstract	The document describes how to perform RF testing using FR1 FR2 Test Tool.

Contents

1 Introduction

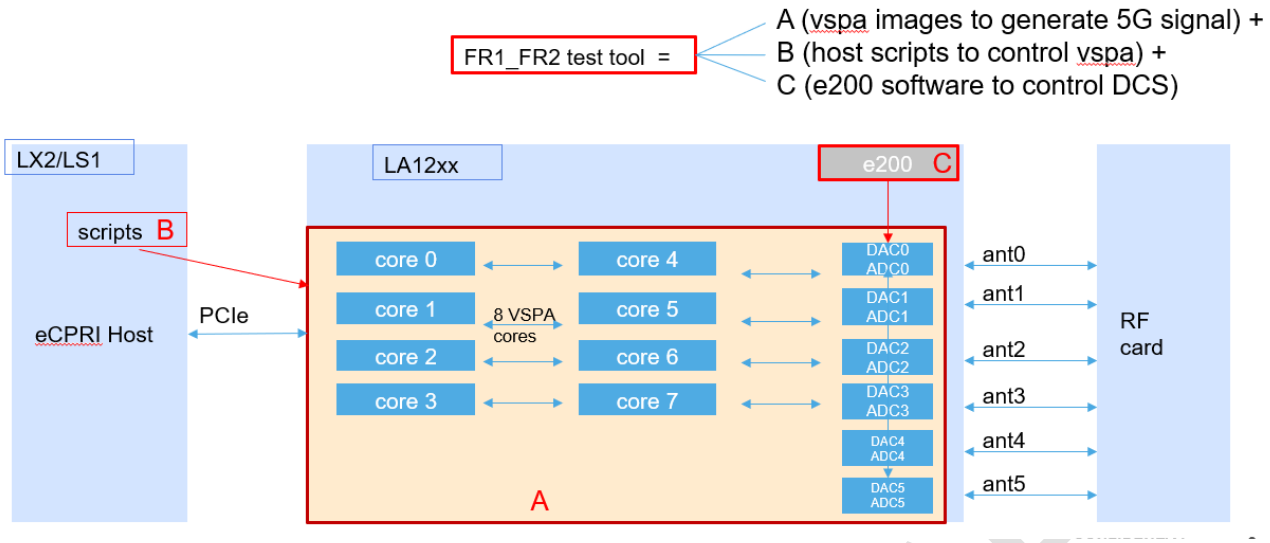
FR1 FR2 Test Tool is a software tool running on NXP LX/LS/iMX + LA12xx/LA93xx device. It functions as a 5G signal generator/receiver for sub 6G (FR1) or mmwave (FR2), supporting different bandwidth, sub-carrier spacing, sampling rate, with a bunch of debug/test/calibration features.

1.1 Features

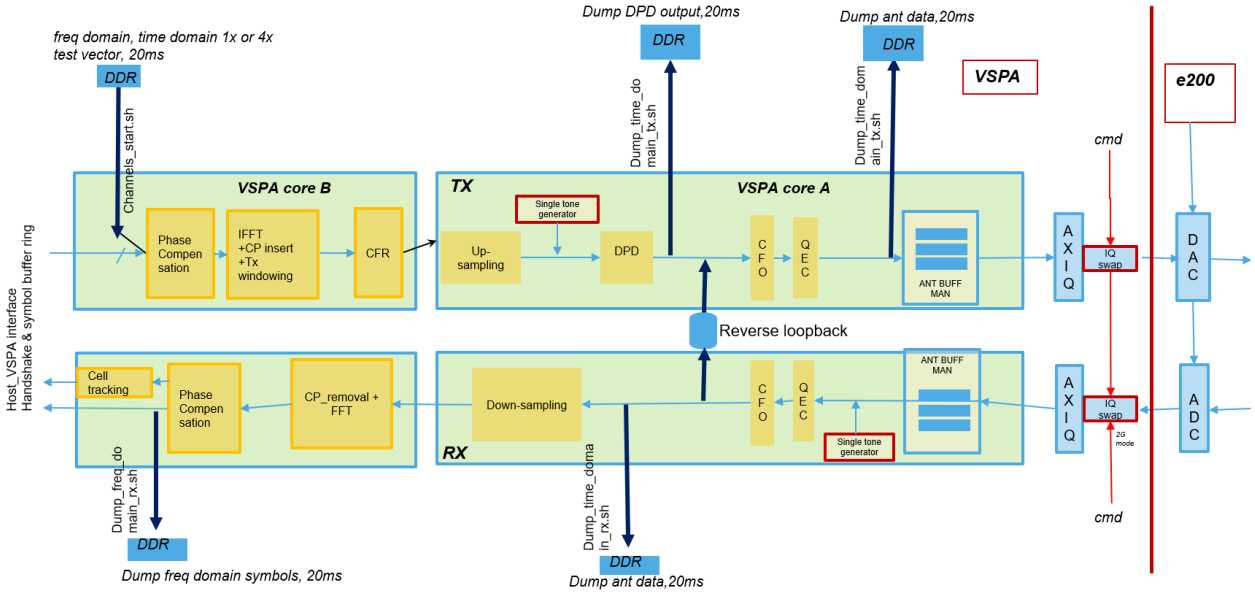
- LSDCS 4T4R on LA12xx, up to 100Mhz, DAC 491-61Msps, ADC 245-61Msps
- HSDCS 2T2R on LA12xx, up to 400Mhz, DAC/ADC 1.9Gsps and 983Msps
- HSDCS 1T1R on LA12xx, up to 1600Mhz, DAC/ADC 1.9Gsps
- FDD 1T1R or TDD 1T2R on LA93xx, up to 25Mhz, DAC 61Msps, ADC 61Msps
- TDD and FDD for each case
- Base-station TDD mode and CPE TDD mode.
- Source waveform can be frequency domain, time domain 1x (option8), time domain 4x (up-sampled),
- Waveform length can be 0.5ms to 40ms
- Time domain single tone (embedded tone generator, no waveform needed), multiple single tones.
- Frequency domain single sub-carrier
- IQ SWAP & PN SWAP
- QEC open & close loop training
- DPD open & close loop training on LS or HS observation path
- User defined DPD model
- DPD model auto search
- CFO compensation
- phase compensation
- Timing offset compensation
- Cell tracking
- SINAD measurement
- Signal amplitude scaling
- Reverse loopback from ADC back to DAC
- Memory DMA copy over PCI latency test
- dump time domain antenna data
- dump frequency domain symbol data

The test tool contains 3 parts:

- A: VSPA images running on LA12xx or LA93xx VSPA cores.
- B: Scripts running on host processor
- C: DCS and RF control logic running on LA12xx e200 core or LA93xx M4 core



1.2 Test Tool Block Diagram:

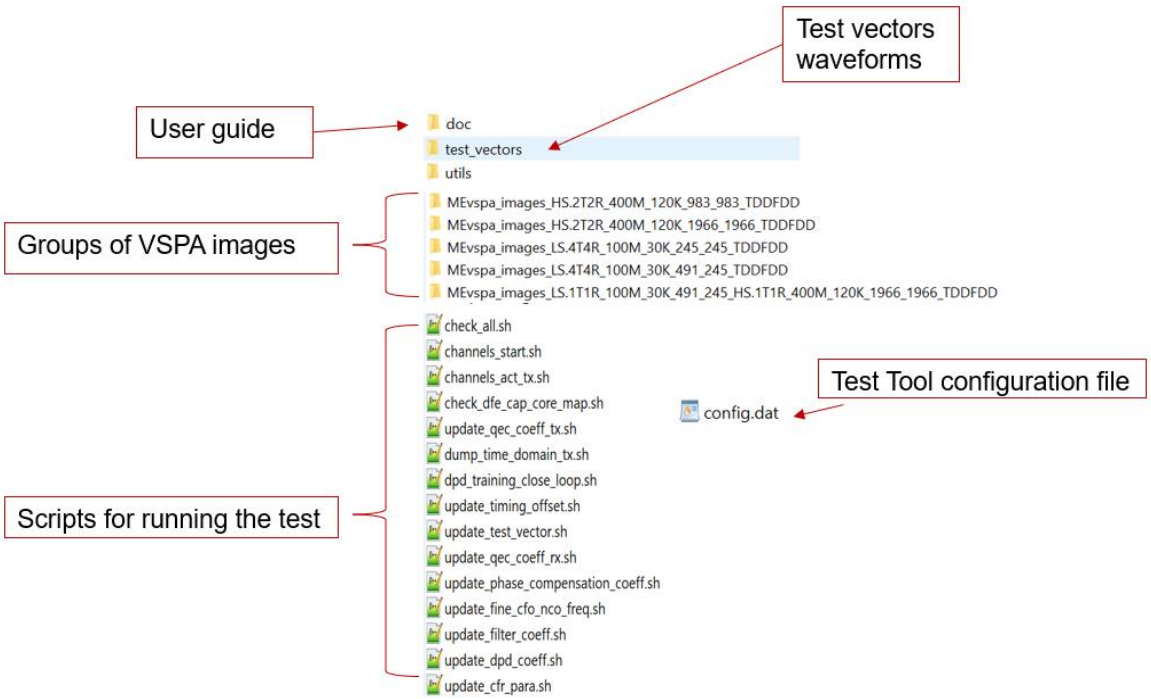


For FR2, there are no CFR and DPD modules.

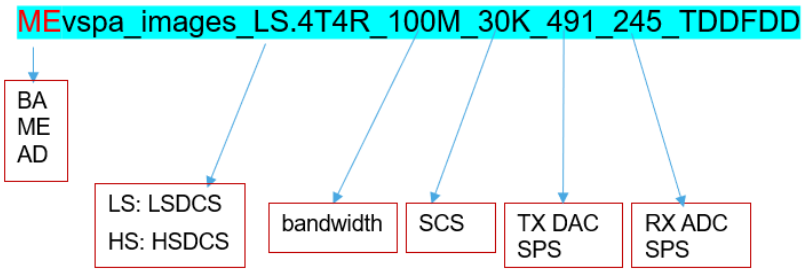
1.3 Test Tool Package

The test tool software packages contains:

- VSPA images
- Scripts for running the test.
- User guide



VSPA images naming:



BA: basic version, all functionalities enabled except QEC, DPD, CFR.

ME: medium version, all functionalities enabled except DPD, CFR.

AD: advanced version, all functionalities enabled

1.4 Install the Test Tool

Download the Test Tool from NXP Docstore, Users need request permission from NXP for first time.

Untar the package fr1_fr2_test_tool_v5.1.tgz to NXP LS/LX/iMX device under directly fr1_fr2_test_tool/,

Go to fr1_fr2_test_tool/ directly where you can see scripts such as boot_vspa.sh

```
root@localhost:~/michael/fr1_fr2_test_tool# ls boot_vspa.sh -l
-rwxr-xr-x 1 root root 14874 Nov  4 05:52 boot_vspa.sh
root@localhost:~/michael/fr1_fr2_test_tool#
```

All commands should be run under this directory.

2 Prepare for Testing

2.1 Choose VSPA image

Test tool provides a group of VSPA images for different test cases, users should choose desired image for their test case according to DCS sampling rate and bandwidth.

Typical image used for FR1 on LSDCS is ADvspa_images_LS.4T4R_100M_30K_491_245_TDDFDD.

Typical image used for FR2 on HSDCS is MEvspa_images_HS.2T2R_400M_120K_1966_1966_TDDFDD.

Image name (running on LA12xx)	test cases
ADvspa_images_LS.4T4R_100M_30K_491_245_TDDFDD ADvspa_images_LS.4T4R_50M_30K_245_122_TDDFDD ADvspa_images_LS.4T4R_25M_30K_122_122_TDDFDD ADvspa_images_LS.4T4R_10M_30K_61_61_TDDFDD	Up to 4T4R test on LSDCS Any of the 4 RX channels can be used as DPD loopback path. NOTE: For TDD, 4T4R can work normally. For FDD in 100Mhz case, 4T or 4R or 2T2R can work normally, 4T4R may not work. The four images provide different bandwidth and different ADC/DAC sampling rates
MEvspa_images_HS.2T2R_400M_120K_1966_1966_TDDFDD	Up to 2T2R test on HSDCS
ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD ADvspa_images_LS.2T2R_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD_DPD19	Up to 2T2R test on LSDCS and 2R test on HSDCS This image is mainly used for DPD test, input waveform must be time domain 4x up-sampled 491.52Msps. The LS RX and HS RX can be used as DPD feedback path. The two images are using different DPD models
ADvspa_images_LS.2T2R_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD	Same as above, the difference is that the input waveform is frequency domain or time domain at 122.88Msps(option8)
MEvspa_images_HS.1T1R_800M_120K_1966_1966_TDDFDD	1T1R 800Mhz test on HSDCS Input waveform must be time domain 983Msps
MEvspa_images_HS.1T1R_1600M_120K_1966_1966_TDDFDD	1T1R 1600Mhz test on HSDCS Input waveform must be time domain 1966Msps with length 0.5ms.
vspa_images_LS.4T4R_100M_30K_245_245_HS.2T2R_400M_120K_983_983_TDDFDD_SINAD vspa_images_LS.4T4R_100M_30K_491_245_HS.2T2R_400M_120K_1966_1966_TDDFDD_SINAD	Used for SINAD measurement Only single tone supported. single tone can be sent from any of the 6 DACs. SINAD measurement can be done on any of the 2 HSADCs.
ADvspa_images_LS.4T4R_100M_30K_491_245_TDDinFDD	TDD in FDD mode, DCS works in FDD mode while software is TDD mode.
ADvspa_images_LS.4T4R_100M_30K_491_245_TDDFDD_ISC	Using 4 VSPA cores for DFE, other 4 cores reserved for HIghPHY. (internal use only)

NOTE: All images with tag 1966 supports 983Msps also, refer to ./boot_vspa.sh options.

Image name (running on LA9310)	test cases
MEvspa_images_LS.1T2R_10M_15K_61_61_TDDFDD_LA93	LA9310 10Mhz SCS 15Khz test

MEvspa_images_LS.1T2R_25M_30K_61_61_TDDFDD_LA93	LA9310 25Mhz SCS 30Khz test
MEvspa_images_LS.1T2R_25M_60K_61_61_TDDFDD_LA93	LA9310 25Mhz SCS 60Khz test, cell tracking disable.
MEvspa_images_LS.1T1R.T4x_25M_60K_122_122_TDDFDD_LA93	LA9310 122Msps test, only single tone supported

2.2 Check Configuration File

Test tool provides a configuration file named config.dat to keep the default configurations. Experienced users can change the configurations according to their test case. Beginner users should use the default configurations.

configurations	Default values	Other possible values and use cases
TDD/FDD	tx_fdd=1 rx_fdd=1 FDD mode	tx_fdd=0 rx_fdd=0 : TDD tx_fdd=0 rx_fdd=1 : TX TDD, RX FDD. Used in external loopback TX to RX, RX can dump TX TDD signal
DCS channels to enable	DCSchan=(1 1 1 1 1 1) All 6 DCS channels enabled	Any of the channels can be set to 0
TDD pattern	pattern0=(7 6 2 4 0 0) TDD pattern DDDDD DDSUU	Any TDD patterns are possible
eCPRI compression	ecpri_comp_decomp_enable=0	Set to 1 to enable eCPRI compression
option8 or ecpri	option8=0 Use frequency domain waveform	Set to 1 to use time domain waveform
IQ SWAP	iqswap_tx=(0 0 0 0 0 0) iqswap_rx=(0 0 0 0 1 0) IQ SWAP enabled on HSADC0, which is required due to hardware limitation	Any bit can be set to 1 or 0
DCS enabling	use_nxp_l1c=1 Use NXP provided L1C to enable DCS	Set to 0 if users want to use their own TBGEN code to enable DCS
waveform length	input_waveform_len=(20 10) 20ms for TDD, 10ms for FDD	0.5-40ms

2.3 Check e200 image (LA12xx only)

If users are testing FDD mode, they can use default yami.ko and default e200 image (FDD is irrelevant to e200 image).

If users are testing TDD mode, they should use /lib/firmware/geul_e200_rudemo.elf or their own e200 image which support control of AXIQ/DCS to work in TDD mode

If users rebuild e200 image, they can build with RUDEMO flag ON (RUDEMO=1) to support TDD.

Test tool provides an option to choose which e200 image to use (default is geul_e200_rudemo.elf) by passing e200 image path and name to ./boot_vspa.sh)

3 Start the Testing

3.1 Two steps to start the testing:

STEP1: Boot VSPA image

./boot_vspa.sh <vspa_image>

- Users should change the path of yami.ko if their path is different from the one in the script
- The script will automatically copy the VSPA images in the specified folder to /lib/firmware and boot them
- The script will set up DAC/ADC clock automatically according to the naming policy of the image folder
- The script will show boot successful and prompt next step

STEP2: Start Channels

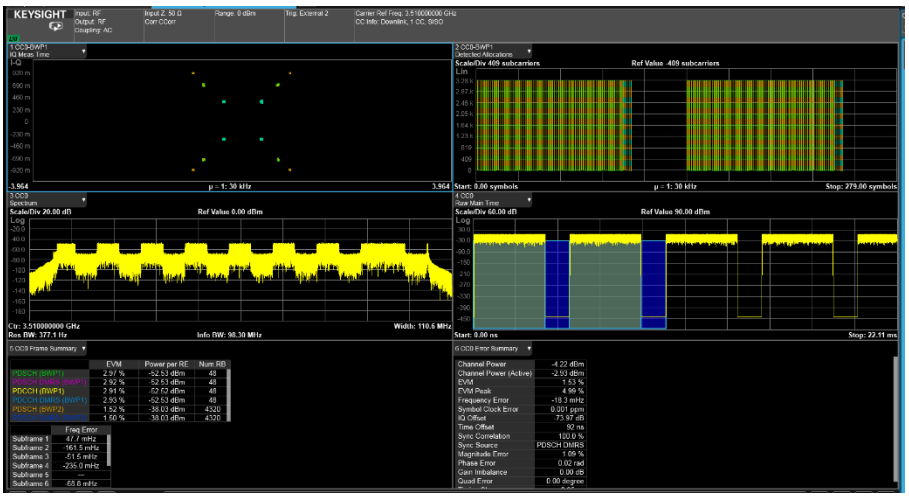
./channels_start.sh [dcsX]

- *This step will start specified DCS channels or all LSDCS and all HSDCS channels, using default configurations in file config.dat.*
- *The script will talk to the VSPA images and know which images are used, and load relevant test vectors, enable relevant DCS channels automatically if use_nxp_l1c=1, or wait for users to enable DCS manually if use_nxp_l1c=0*
- *dcsX is the DCS channel that users want to use, such as dcs0 for LSADC0/DAC0.*
- *After this step, signal is expected to be sent out to DAC and users can start testing RF signal.*

If your device part number is HSDCS de-featured, DO NOT start HS channels.

Now, users can see TM signals on equipment

If everything is working fine, the expected signal waveform can be seen at DAC output, RFIC, PA or antenna, like what the picture shows.



If no signal is seen or signal is bad, the reasons could be

- Test tool issue
- RF card Hardware issue
- Test equipment setup issue

To identify whether it's test tool issue, follow the following sections.

3.2 Check Test Tool Status

After step2, the console will print test tool information and channel status information like below.

RUNNING/NORMAL means the channel is working correctly. Otherwise if the status is STOPPED, the channels is NOT working.

Num symbols sent/received, which should keep increasing. Run ./check_all.sh to check status again to get the latest numbers sent/received

```
*******************************************************************************
***                              DFE capability
***                    FR1 capabilities:
***                        Product Type:   RU TEST_TOOL
***                                 CFR:   ENABLED, num of PASS = 4
***                                 DPD:   ENABLED @491520 KSPS, DPD CONFIG ID = 4, num coeff = 28
***                           DPD model:   [ 0 0 5 ][ 1 0 2 ][ 0 1 4 ][ 1 1 4 ][ 0 2 4 ][ 2 2 3 ]
***        DPD 4T4R close loop training:   DISABLED
***                  up-sampling filter:   64 taps, 4x interpolation
***                               TXQEC:   ENABLED
***                               RXQEC:   ENABLED
***                                iFFT:   ENABLED
***                                 FFT:   ENABLED
***                          eCPRI Decomp:   ENABLED
***                            eCPRI Comp:   ENABLED
***                    SINAD measurement:   DISABLED
***                Channels running time:   hh:mm:ss 0:0:11
*******************************************************************************
*****************************  ANT ID 0: TX on CORE4&0  ***********************************************
***                            Bandwidth:   TX: 100 Mhz, SCS 30 Khz
***                     DCS channel used:   TX: FDD LSDCS0-0   491520  KSPS
***            Num symbols sent/received:   TX: 0x000031a9
***                                Status:   TX: RUNNING
***                               TX MODE:   Freq domain waveform @100 Mhz, filename: ./test_vectors/TM1.1_100MHz_30kHz_FDD.bin
***              tx_allowed current state:   HIGH
***           DCS first enabled timestamp:   tx_allowed 0x8a13efd3
***                            Kernel load:   iFFT       : 9%
***                            Kernel load:   CFR        : 0%
***                            Kernel load:   UP sampling: 20%
***                            Kernel load:   DPD        : 50%
***                            Kernel load:   QEC    6%
***                             CFR status:   OFF NPASS = 0
***                             DPD status:   ON
***                             QEC status:   ON
***                          PNSWAP status:   OFF
*******************************************************************************************************
FR1 FR2 Test Tool  Version:   4.0
Current VSPA image Version:   4.0.5

Number of enabled channels TX: 4,  RX: 4
Number of Running channels TX: 4,  RX: 4
MODE: TX FDD, RX FDD,   Basestation

Ant Mapping TX:
ant_id  dcsid     dcs           status
0       0         LSDCS0-0    RUNNING
1       1         LSDCS0-1    RUNNING
2       2         LSDCS1-0    RUNNING
3       3         LSDCS1-1    RUNNING

Ant Mapping RX:
ant_id  dcsid     dcs           status
0       0         LSDCS0-0    RUNNING
1       1         LSDCS0-1    RUNNING
2       2         LSDCS1-0    RUNNING
3       3         LSDCS1-1    RUNNING
```

Items to check: SW version, product type, bandwidth&DCSrate, status TX, waveform file name, num running channels, ant_id dcs_id mapping. Specifically, check the DCS channel to see whether your RF card and

equipment is using the same DCS channel. And the channel status is RUNNING.

Ant_id vs dcsid

- The test tool VSPA images support 4T4R for LS and 2T2R for HS, and in some images support 2T2R for LS. Channel ID or ant_id is used to identify which 1T1R is used.
  - For 4T4R LS, ant_id is from 0 to 3
  - For 2T2R HS, ant_id is from 4 to 5
  - For 2T2R LS, ant_id is from 0 to 1
  - For 1T1R HS, ant_id is fixed to 4
- There are 6 DCS channels in LA12xx device, DCS ID is used to identify which DCS channel is used.
  - DCS ID 0-3: LSDCS0-3, DCS ID 4-5: HSDCS0-1
- Mapping between ant_id and DCS ID
  - By default, ant_id 0 is mapped to dcs_id 0 and so on.
  - Users can re-map ant_id to other dcs_id if wanted. For example, users is using 2T2R image and want ant_id 0 to map to dcs_id 2. run ./channels_start.sh dcs2
  - Ant_id is used in all scripts to identify which channel the script will operation on. Users can find the ant_id dcs_id mapping in the log of channels_start.sh, or run ./check_all.sh which will show the mapping.

3.3 Check ant data dump

To make sure test tool is sending correct data to DAC, run the following command to dump the data at DAC.

./dump_time_domain_tx.sh [ant_id]

- This script will dump 20ms TX ant data to a file, and verify it's md5sum, if the dumped ant data are all expected, the script will print CORRECT, *DAC is sending expected signal*
- Users can use Matlab or test equipment to analyze the data

If status TX is running, and this script prints "CORRECT! CORRECT! CORRECT! ***** Ant data sent to DAC are expected, it means Test Tool itself is working well and sending correct antenna data out to DAC, users should focus on RF card hardware issue or equipment setup issue if signal is not seen or signal is bad.

Otherwise the test tool does not send correct data to DAC, users should check the LA device/board/BSP version/boot log etc for possible reason, or send all test log to NXP local support for help.

NOTE: if users change the default configurations in config.dat, or they don't use default waveform file, it may not print CORRECT, users should analyze the dumped data and judge whether the dumped data are expected.

3.4 Boot VSPA options

./boot_vspa.sh <vspa_image> [e200_image] [hsdiv2]

Options	Possible values	comment
vspa_image	One of the VSPA image folder name provided in test tool	VSPA image folder name. Mandatory.
e200_image		e200 image path and name. optional. If not specified, /lib/firmware/geul_e200_rudemo.elf will be used.
hsdiv2		To run HSDCS in 983Msps. Optional. Examples: **./boot_vspa.sh MEvspa_images_HS.2T2R_400M_120K_1966_1966_TDDFDD hsdiv2** This will set both TX and RX of the 2T2R channels to run at 983Msps **./boot_vspa.sh ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD hsdiv2** This will set RX channels of the 2R to run at 983Msps.

3.5 Channels Start options

./channels_start.sh has provided the following options to enable users to control the channel flexibly.

All options are optional. Options can be put in any order.

./channels_start.sh [op1] [op2] … [opn]

Options	Possible values	comment
0 1 2 3 4 5		Ant_id, start specified ant ids. If not specified, start all channels that VSPA image provides. Example: to start only channel 0 (ant_id 0): ./channels_start.sh 0 to start channel 0 and 2: ./channels_start.sh 0 2 Usually, users do not need to specify channel IDs, let test tool automatically allocate channels. Limitation: if ant_id specified, the same dcs_id will be used, for example, ./channels_start.sh 2 will start ant_id 2 and dcs_id 2.
dcsX, txdcsX, rxdcsX	X can be 0-5	start specified dcs_id. dcsX is equivalent to txdcsX plus rxdcsX. Example: to start TX and RX on dcs2: ./channels_start.sh dcs2 to start TX on dcs1 but RX on dcs2: ./channels_start.sh txdcs1 rxdcs2 note that ant_id will be automatically mapped to the dcs_id.
tdd,fdd, txtdd,txfdd, rxtdd,rxfdd		tdd: set both TX and RX to TDD, fdd: set both TX and RX to FDD txtdd, txfdd: set TX to TDD or FDD rxtdd, rxfdd: set RX to TDD or FDD
txonly, rxonly		Set all channels to TX ONLY or RX ONLY
option8		Set to use time domain waveform
filename		Set to use specified waveform file

		Example: to use abcd.bin as input waveform: ./channels_start.sh abcd.bin
mimo		Different channels use different waveform file. If not specified, all TX channels will use the same waveform file. Example: to use abcd0/1/2/3.bin to channel 0/1/2/3: ./channels_start.sh abcd0.bin mimo ./update_test_vector.sh 1 abcd1.bin ./update_test_vector.sh 2 abcd2.bin ./update_test_vector.sh 3 abcd3.bin
HRAM		To put input waveform on HRAM. By default input waveform is put on DDR, in case PCI bandwidth is not high enough, specify this option to put waveform on HRAM. As HRAM is small, waveform length should be reduced.
0.5ms, 1ms, 2ms,5ms, 10ms, 20ms,40ms		Specify waveform length. If not specified, default waveform length defined in config.dat will be used. Example: to use short waveform on HRAM: ./channels_start.sh 0.5ms HRAM
cpe		set test tool to work as CPE/UE mode. In TDD, RX will start first.
nre=X nrb=Y	X,Y can be any integer numbers	Number of RE, or number of RB in a frequency domain symbol. To run with reduced bandwidth, such as run 60Mhz on 100Mhz channel, specify this option.
rxinj=filename		Run the RX channel with data from specified waveform instead of ADC. If this option is used, RX channel will not run in 5G timing at ADC sampling rate. Example: when users take RX time domain dump from field, and hope to reproduce the same situation, run ./channels_start.sh dcs0 rxonly rxinj=abcd.bin
nl1c		Do not use NXP l1c to enable DCS. Use this option if users have their own way to enable DCS.
idle		To start channels with DAC sending all 0.
restart		Use this option if users want to change options or mode. Example: if users want to change from FDD mode to TDD mode. ./channels_start.sh dcs0 fdd after FDD mode testing is done, ./channels_start.sh dcs0 tdd restart
loopback, lb		Set HSDCS to work in digital loopback mode
rlbt		Reverse loopback from time domain, this option will enable users to get received ADC data being sent back to ADC. Received data will be put into a buffer and TX will send the data from the same buffer. There is a delay between TX and RX. The requires ADC and DAC works in the same sampling rate and in FDD mode
hwdcm nhwdcm		hwdcm Sets HSADC to work in hardware decimation mode. In this mode, ADC sampling rate will be reduced by 2x. nhwdcm will disable the hardware decimation.
fast		With this option, test tool commands will run faster by reducing un-necessary prints or actions.

3.6 TX Input waveforms

- Test tool supports 3 types of input waveforms with length 0.5-40 ms
  - Frequency domain waveform
    - Start channels with ./channels_start.sh

  - Time domain waveform (option8)
    - 122.88Msps for 100Mhz, 491.52Msps for 400Mhz
    - ./channels_start.sh option8
  - Time domain up-sampled waveform (up-sampled to DAC sampling rate)
    - 491.52Msps for LSDAC
    - Only specific VSPA image supports (ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD)

- PCI bandwidth requirement if waveform is on host side DDR
  - To support above waveforms, PCI bandwidth requirement is as below table
  - If users do not have enough PCI bandwidth, they can choose to use HRAM to hold the waveform file. In this case waveform length is limited to HRAM size which is 6MB.

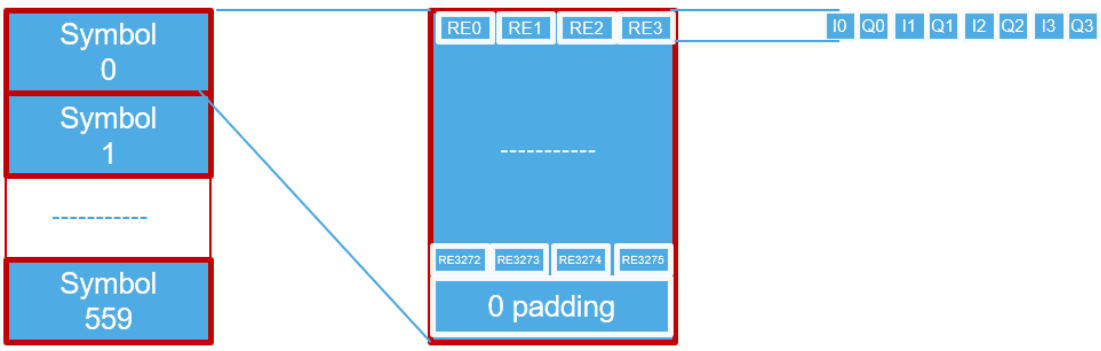
cases	Freq domain	time domain non-upsampled	time domain upsampled
LSDAC@491Msps	3Gbps	4Gbps	16Gbps
HSDAC@1.9Gsps	12Gbps	16Gbps	64Gbps

3.7 Waveform format

Frequency domain waveform format:

Frequency domain waveform consists of a certain number of frequency domain symbols. Each symbol consists of a certain number of samples (RE). The size of each symbol is zero padded to 128 bytes aligned. Each symbol has a 16-bit real part and a 16-bit imaginary part (IQ sample) in 2's complement format. Each 16 bit I or Q is stored in little endianness.

Example, for 20ms length waveform for 100Mhz SCS30Khz use case, the number of symbols is 560, the number of RE in each symbol is 3276, the format of the waveform is illustrated below:



hexdump ./test_vectors/TM1.1_100MHz_30kHz_FDD.bin | more

```
0000000 16a0 16a0 e960 e960 16a0 16a0 16a0 16a0        #symbol 0 starts
0000010 e960 16a0 e960 16a0 e960 16a0 16a0 16a0
0000020 16a0 16a0 16a0 e960 e960 e960 16a0 16a0
0000030 16a0 16a0 16a0 16a0 16a0 16a0 e960 16a0
……
0003320 16a0 e960 16a0 e960 e960 e960 16a0 16a0
0003330 0000 0000 0000 0000 0000 0000 0000 0000        #pad zeros to 128 bytes aligned address
0003340 0000 0000 0000 0000 0000 0000 0000 0000
0003350 0000 0000 0000 0000 0000 0000 0000 0000
0003360 0000 0000 0000 0000 0000 0000 0000 0000
0003370 0000 0000 0000 0000 0000 0000 0000 0000        #symbol 0 ends
0003380 e960 16a0 16a0 16a0 16a0 16a0 16a0 16a0        #symbol 1 starts
……
```

4 Runtime Commands

After channels are started, users can run different commands to control test tool to perform various actions.

The following sections list some often used commands. For all the commands please refer to the test tool package scripts.

4.1 Update TX Waveform

Command syntax: ***./update_test_vector.sh [ant_id] [waveform _filename]***	
Options	***meaning***
Ant_id	***0-5, if not specified, update all antennas***
Waveform_filename	***new waveform to use. If not specified, default waveform will be used***
Function: ***This command will update the TX input waveform. TX channels will send new waveform.***	
Example: ***./update_test_vector.sh NR-FR2-TM3.1_100MHz_120kHz_TDD_fd_3168_20ms.bin***	

4.2 Send Single Tone

Command syntax: ***./send_single_tone.sh [ant_id] [freq] [amp] [I|Q] [add] [rx]***	
Options	***meaning***
Ant_id	***0-5, if not specified, 0 will used***
freq	***frequency in Hz***
amp	***amplitude of the single tone. 1-100 representing 1%-100% of DAC full scale. Default is 25% if not specified. If amp is 0, DAC will send all 0's. This can be used to stop sending signal on DAC.***
I|Q	***send on I path only or Q path only, the other path will send all 0***
add	***add current single tone on top of previous single tones. Without add, all previous tone will be removed***

rx	*Generate Single tone on RX channel input. If not specified, generate single tone on TX channel output*
Function: *This command will generate time domain single tone on TX channel output or RX channel input*	
Example: *Send 10Mhz with amplitude 20% on TX ant 0* *./send_single_tone.sh 0 10000000 20* *Send 10Mhz and 20Mhz with amplitude 20% on TX ant 0* *./send_single_tone.sh 0 10000000 20* *./send_single_tone.sh 0 20000000 20 add* *Send 10Mhz with amplitude 20% on RX ant 0* *./send_single_tone.sh 0 10000000 20 rx* *Stop single tone on TX ant 0, and switch to send waveform from file.* *./send_single_tone.sh 0 stop*	
NOTE: *Single tone has much higher average power than 5G TM waveform, users should be careful not to damage PA. Recommend to try with small amp such as 20.*	

4.3 Scale Signal Amplitude

Command syntax: *./scale_percent.sh [ant_id] [percentage] [input] [auto] [dis]*	
Options	*meaning*
Ant_id	*0-5, if not specified, 0 will used*
percentage	*any integer number. Such as 110 will scale signal amplitude to 110% on both I and Q*
input	*Scale the signal on input waveform. If not specified, it will scale the output signal at DAC*
auto	*Automatically scale input and output to get best signal amplitude.*
dis	*Disable the scaling, signal amplitude will go back to original*
Function: *This command will scale up or down the TX signal amplitude on output or input.* *In case input waveform power is very low, should scale input.*	
Example: *Scale output by 125% on TX ant 0* *./scale_percnet.sh 0 125* *Scale input by 125% on TX ant 0* *./scale_percnet.sh 0 125 input* *Automatically scale TX ant 0* *./scale_percnet.sh 0 auto* *Disable scaling on TX ant 0* *./scale_percnet.sh 0 dis*	

4.4 Update Test Vector

Command syntax: *./update_test_vector.sh [ant_id] [filename] [idx=a:b]*	
Options	*meaning*
Ant_id	*0-5, if not specified, will update all antennas*
filename	*New test vector filename. If not specified, will update to original test vector*
Idx=a:b	*Only send specified REs from index a to b, other REs cleared to 0. Num of REs in a symbol is n_re, idx is defined from -n_re/2 to n_re/2-1. For SCS 30Khz, idx 1 stands for subcarrier +30Khz, idx -1 stands for -30Khz.*
Function: *This command will update TX test vector (input waveform) to specified or all antennas.*	
Example: *Update waveform to a.bin for ant 0* *./update_test_vector.sh 0 a.bin* *Send only 10Mhz bandwidth when testing 100Mhz scs 30Khz.* *./update_test_vector.sh 0 idx=-144:143*	

4.5 Update QEC Coefficients

Command syntax: *./update_qec_coeff_tx.sh [ant_id] [filename] [dis]* *./update_qec_coeff_rx.sh [ant_id] [filename] [dis]*	
Options	*meaning*
Ant_id	*0-5, if not specified, 0 will used*
filename	*QEC coefficients file name*
dis	*Disable the updated coefficients, go back to pass-through state*
Function: *This command will update QEC coefficients on TX or RX.*	
Example: *Update QEC coefficients from file abcd.bin on TX ant 0* *./update_qec_coeff_tx.sh 0 abcd.bin* *Disable updated coefficients and set QEC to pass-through on TX ant 0* *./update_qec_coeff_tx.sh 0 dis*	

4.6 Update DPD Coefficients

Command syntax: *./update_dpd_coeff.sh [ant_id] [filename] [dis]*	
Options	*meaning*
Ant_id	*0-5, if not specified, 0 will used*
filename	*DPD coefficients file name*

dis	*Disable the updated coefficients, go back to pass-through state*
Function: *This command will update DPD coefficients on TX.*	
Example: *Update DPD coefficients from file abcd.bin on TX ant 0* *./update_dpd_coeff.sh 0 abcd.bin* *Disable updated coefficients and set DPD to pass-through on TX ant 0* *./update_dpd_coeff.sh 0 dis*	

4.7 Update Phase Compensation Coefficients

Command syntax: *./update_phase_compensation_coeff.sh [ant_id] [filename|addr|arfcn=] [dis] [tx|rx]*	
Options	*meaning*
Ant_id	*0-5, if not specified, 0 will used*
filename	*Phase compensation coefficients file name*
addr	*Memory address from where phase compensation coefficients are stored, must have initial 0x*
Arfcn=	*Specify the arfcn, test tool will generate phase compensation coefficients by this arfcn*
tx|rx	*Update on TX or RX. If not specified, tx will be used*
dis	*Disable the updated coefficients, go back to pass-through state*
Function: *This command will update phase compensation coefficients on TX or RX*	
Example: *Update phase compensation coefficients from file abcd.bin on TX ant 0* *./update_phase_compensation_coeff.sh 0 abcd.bin* *Update phase compensation coefficients by arfcn on RX ant 0* *./update_phase_compensation_coeff.sh 0 arfcn=2 rx*	

4.8 Dump Time Domain Antenna Data

Command syntax: *./dump_time_domain_tx.sh [ant_id] [len] [dpdo] [offset=x] [pow]* *./dump_time_domain_rx.sh [ant_id] [len] [offset=x] [pow]*	
Options	*meaning*
Ant_id	*0-5, if not specified, 0 will used*
len	*len can be one of the following types* *len<8192, dump data length will be len*32768 bytes* *len>=8192, dump data length will be len bytes* *0.5ms|1ms|2ms|5ms|10ms|20ms|40ms*
dpdo	*Dump DPD output data. If not specified, dump ant data sent to DAC*

[offset=x]	*Specified the starting point from frame boundary. x=0x1-0x7F, standing for num of 64KB (16K samples)*
pow	*Measure the power of the dumped data. The sample power dbfs, max I or Q values will be printed. If no power information is printed, reduce the dump size.*
Function: *This command will dump ant data sent to DAC or received from ADC, or dump the DPD output data, dumped data will be stored to a file*	
Example: *Dump DPD output data 128KB from TX ant 0* *./dump_time_domain_tx.sh 0 4 dpdo* *Dump ant data 20ms from TX ant 0* *./dump_time_domain_tx.sh 0 20ms* *Dump ant data 1ms from RX ant 0 with an offset of 64KB from frame boundary* *./dump_time_domain_rx.sh 0 1ms offset=1*	

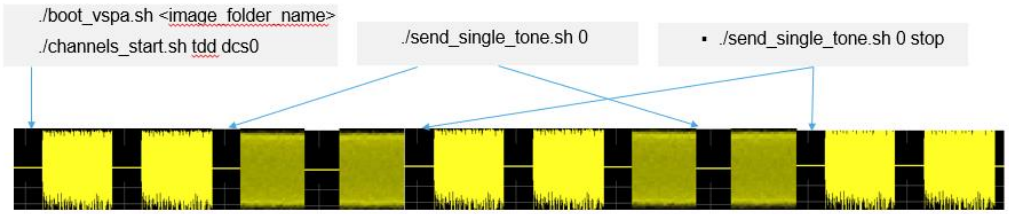
4.9 Dump Frequency Domain Symbol data

Command syntax: *./dump_freq_domain_tx.sh [ant_id] [len]* *./dump_freq_domain_rx.sh [ant_id] [len]*	
Options	*meaning*
Ant_id	*0-5, if not specified, 0 will used*
len	*0.5ms|1ms|2ms|5ms|10ms|20ms|40ms. If not specified, default len will be used (FDD 10ms, TDD 20ms)*
Function: *This command will dump frequency domain symbols, dumped data will be stored to a file. If the vspa image is working in time domain mode(T4x) or option8 mode, or only supported time domain waveform, dump frequency domain will result in dumping time domain. Pay attention to the dumped file name.*	
Example: *Dump 20ms from TX ant 0* *./dump_freq_domain_tx.sh 0 20ms*	

5 Test Case Examples

5.1 General TX/RX Test

By default, after channels are started, TM signal will be sent to DAC. Users can send single tone at any time and can switch between TM signal and single tone.

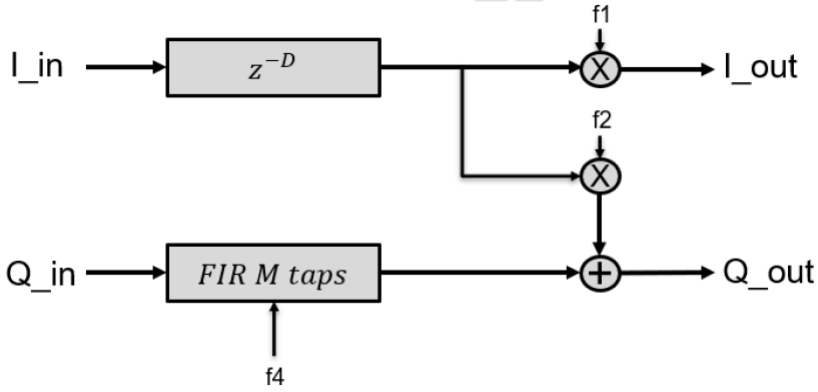


At any time, users can run ./dump_time_domain_tx.sh to dump the antenna data sent to DAC.

At any time, users can run ./dump_time_domain_rx.sh to dump the antenna data received from ADC.

5.2 QEC Calibration Test

- QEC will do the following on TX and RX ant data
  - IQ imbalance compensation
  - Gain compensation
  - DC offset compensation
  - Timing skew compensation (Delay between the I and Q. Not all VSPA images. Currently only ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_983_983_TDDFDD supports with filter taps from 6 to 16)
- QEC coefficient data structure
  - Users should generate QEC coefficients in the following structure, and feed the coefficients to test tool scripts ./update_qec_coeff_tx/rx.sh



QEC coefficients struct:

```
typedef struct{
    cfixed16_t  state[32];
    cfloat32_tFD_taps[16];
    uint32_t int_del;
    uint32_t FD_tap_cnt;
    float32_t f1;
    float32_t f2;
    float32_t  f4;
    cfloat32_t g;
    cfloat32_t dc;
}qec_params_t;
```

For detail of QEC open loop and close loop calibration, please refer to *UG10096.PDF QEC & DPD Training User Guide.*

5.3 DPD Training Test

- The DPD Algorithm
  - $y(n) = \sum_{r,q} \sum_{p=0}^{P(r,q)} \{ x(n-r) * a_{rqp} * |x(n-q)|^p \}$
  - x:input signal, y:output signal, r:delay of signal, q:delay of signal envelope, P:polynomial, a:complex coefficients.
  - r,q,P are integers, can be any numbers. Users should determine r,q,P values according to their PA.
  - An example DPD model (determined r,q,P values) is shown below, which means when r=0,q=0,p=0~6; when r=1,q=0,p=0~3; etc.

r	q	P
0	0	6
1	0	3
0	1	5
1	1	5

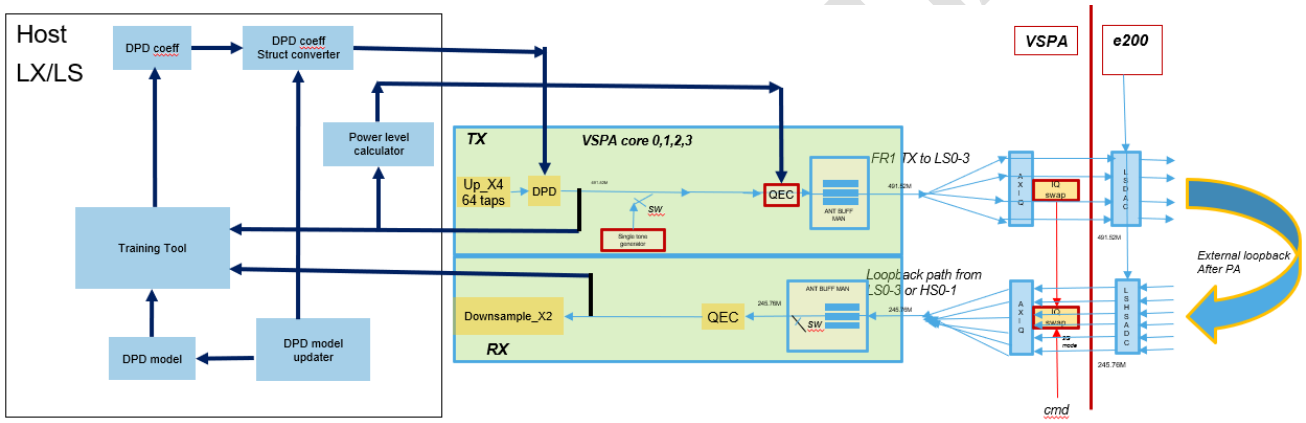
r	q	P
0	0	6
1	0	5
2	0	3
3	0	3
4	0	3
5	0	3
0	1	5
1	1	6
2	1	5
3	1	5
4	1	5
5	1	5
0	2	5
2	2	6
0	3	5
3	3	5
0	4	5
4	4	5
0	5	5
5	5	5

- Different VSPA images may use different DPD models, run ./check_all.sh to check what DPD model is used in the VSPA image.
- Big DPD model
  - Test tool has implemented a highly optimized DPD model with 115 coefficients in the red box in VSPA image
    ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD
  - It supports any customized DPD model with num of coeff equal to or smaller
  - The aim is to enable users to define their own DPD model and test it in test tool, or try different models to see which model is best for their PA
  - See config.dat for how to define customized DPD models
- For detail of DPD open loop and close loop training, please refer to *UG10096.PDF QEC & DPD Training User Guide*

- General steps to test customized DPD models
  - 1. boot vspa using image ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD
  - 2. channels start
  - 3. do QEC on TX path and observation path
  - 4. do DPD close/open loop training multiple iterations to see the performance
  - 5. change DPD model in config.dat
  - 6. repeat step4 and step5 to see performance of different DPD models

5.4 DPD Model Auto Search

It's very difficult for users to find out which DPD model is the best for their PA. To address this issue, Test tool has implemented the following mechanism for DPD model auto search.



The following two VSPA images supports DPD model auto search:

Image folder name	Max Num of Coefficients
ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_983_983_TDDFDD	57
ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_983_983_TDDFDD19	115

Test tool will try DPD models with different num of coefficients within max num of coefficients, find the DPD model with best NMSE.

- DPD model updater will change to next DPD model after current DPD model training is finished according to predefined policy,
- DPD coefficients data struct is converted to match big DPD model required struct.
- In case a bad DPD model is tried, or in case feedback dump is not a good dump (such as external loopback is not done correctly), the coefficients generated by training tool may cause DPD output signal

power to increase to unexpected level and it may damage PA. So before DPD coefficients are applied to DPD actuator, signal output to DAC is shut down in QEC (gain set to 0).

- Power level calculator will monitor DPD output signal power level, if the power level after coefficients are applied is exceeds expected level, it will cancel the coefficients by setting DPD to passthrough state.

For detail of DPD model auto search, please refer to *UG10096.PDF QEC & DPD Training User Guide*

5.5 RFIC Calibration Test

Users can send different time domain sequence to TX path and loopback from after RFIC to RX path, by dumping the RX time domain data users can analyze the RFIC attributes and get RFIC calibration parameters.

Example of steps for RFIC calibration on 1 channel (assume LS DCS 0-0):

1. Prepare time domain sequences with specific length, such as 0.5ms at 491.52Msps, stored as bin file, such as rfic_cal_seq_X.bin (X=1-n, assume users have n time domain sequences).
2. Run ./boot_vspa.sh ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD
3. Run ./channels_start.sh dcs0 rfic_cal_seq_1.bin 0.5ms
4. Run ./dump_time_domain_rx.sh 0.5ms
5. Analyze the dump file
6. Change to next time domain sequence, run ./update_test_vector.sh rfic_cal_seq_2.bin, go back to step 4 until all sequences are done.

Example of steps for RFIC calibration on 2 channels sequentially (assume LS DCS 0-0 and 0-1):

1. Run ./boot_vspa.sh ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD
2. Run ./channels_start.sh dcs0 dcs1 idle rfic_cal_seq_1.bin 0.5ms
3. Run ./channels_act_tx.sh 0
4. Run ./dump_time_domain_rx.sh 0 0.5ms
5. Analyze the dump file
6. Run ./channels_act_tx.sh 1
7. Run ./dump_time_domain_rx.sh 1 0.5ms
8. Analyze the dump file
9. Change to next time domain sequence, run ./update_test_vector.sh rfic_cal_seq_2.bin, go back to step 3 until all sequences are done.

Example of steps for RFIC calibration on 2 channels simultaneously (assume LS DCS 0-0 and 0-1):

NOTE: This requires higher PCI bandwidth, if PCI bandwidth is not high enough, underrun may occur, in this case users can try to put option HRAM as ./channels_start.sh argument.

1. Run ./boot_vspa.sh ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD
2. Run ./channels_start.sh dcs0 dcs1 idle rfic_cal_seq_1.bin 0.5ms HRAM
3. ./dump_time_domain_rx.sh 0 +1 0.5ms
4. Analyze the two dump files
5. Change to next time domain sequence, run ./update_test_vector.sh rfic_cal_seq_2.bin, go back to step 3 until all sequences are done.

Example of steps to calibrate 4 channels

Option1: calibrate 2 channels each time:

1. Run ./boot_vspa.sh ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD
2. Run ./channels_start.sh dcs0 dcs1 rfic_cal_seq_1.bin 0.5ms HRAM
3. ./dump_time_domain_rx.sh 0 +1 0.5ms
4. Analyze the two dump files

5. Change to next time domain sequence, run ./update_test_vector.sh rfic_cal_seq_2.bin, go back to step 3 until all sequences are done.
6. Run ./channels_start.sh dcs2 dcs3 rfic_cal_seq_1.bin 0.5ms HRAM restart
7. ./dump_time_domain_rx.sh 0 +1 0.5ms
8. Analyze the two dump files
9. Change to next time domain sequence, run ./update_test_vector.sh rfic_cal_seq_2.bin, go back to step 7 until all sequences are done.

Option2: calibrate 4 channels simultaneously:

1. Prepare time domain sequences with specific length, such as 0.5ms at **122.88** Msps, stored as bin file, such as rfic_cal_seq_X.bin (X=1-n, assume users have n time domain sequences).
2. ./boot_vspa.sh ADvspa_images_LS.4T4R_100M_30K_491_245_TDDFDD
3. ./channels_start.sh option8 0.5ms rfic_cal_seq_1.bin HRAM
4. ./dump_time_domain_rx.sh 0 +1 +2 +3 0.5ms
5. Analyze the 4 dump files
6. Change to next time domain sequence, run ./update_test_vector.sh rfic_cal_seq_2.bin, go back to step 4 until all sequences are done.

Fast calibration

To make test tool run faster, users can specify option 'fast' to channels_start.sh.
./channels_start.sh dcs0 dcs1 idle rfic_cal_seq_1.bin 0.5ms HRAM fast
NOTE: With this option, test tool will not check errors, suggest users to use this option only after they have tested all steps successfully and steadily without seeing any errors.

Another option is using option 'mem' to dump data into memory only instead of dumping to file.
./dump_time_domain_rx.sh 0 +1 +2 +3 0.5ms mem
Above command will show the following information, and users can use the memory address and size to get the dump data directly from memory.

Synced RX dump for 4 antennas: 0 1 2 3
Antenna 0 dumping done, size:491520, address:0xa06d000000
Antenna 1 dumping done, size:491520, address:0xa06d078000
Antenna 2 dumping done, size:491520, address:0xa06d0f0000
Antenna 3 dumping done, size:491520, address:0xa06d168000

5.6 Reverse Loopback Test

Reverse loopback is done in software, when IQ data is received by ADC, the data will be sent to DAC directly. This enables users to send a signal from equipment from receive antenna and the signal will be sent back to TX antenna. In this test, the DAC and ADC sampling rate must be same. As the received IQ data is buffered in memory, there is a delay between TX signal and RX signal. The exact delay is printed in channels_start.sh log.

In reverse loopback, only commands for QEC, CFO, scaling, time domain dump are valid.

Steps:

./boot_vspa.sh <vpsa_image>
./channels_start.sh <dcsX> rlbt

5.7 Play Customized Time Domain Waveform

If users want to play a customized time domain waveform, they can choose appropriate VSPA image to play it.

On LSDCS, choose:
ADvspa_images_LS.2T2R.T4x_100M_30K_491_245_HS.2R_400M_120K_1966_1966_TDDFDD
This image can play any 491.52Msps time domain waveform with length from 0.5ms to 20ms, bandwidth and sub-carrier spacing can be any values, such as bandwidth from 5Mhz to 200Mhz. Test Tool just reads the waveform and send to DAC. DPD and QEC calibration can be done for this image.

On HSDCS, choose: MEvspa_images_HS.1T1R_1600M_120K_1966_1966_TDDFDD
This image can play any 1966.08Msps time domain waveform with length of 0.5ms, bandwidth and sub-carrier spacing can be any values, such as bandwidth from 5Mhz to 1400Mhz. Test Tool just reads the waveform and send to DAC. QEC calibration can be done for this image.

5.8 IQ SWAP Test

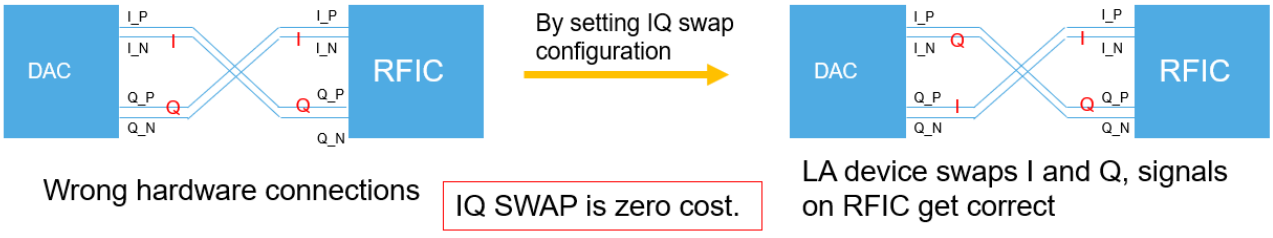
If users made mistake in their hardware design which swaps I and Q on DAC output or ADC input, they need test with IQ swap. IQ swap is done by VSPA to configure AXIQ control register, the hardware will swap IQ according to register value.

To test IQ SWAP:

- Change IQ SWAP configurations in config.dat, see below red box.
- Reboot and start the test
- Users need equipment to measure the DAC output signal to verify IQ SWAP

IQ SWAP can only be enabled at initialization state in config.dat, can not be changed at runtime

```
#IQ SWAP configurations: All TX/RX for all 6 DCS channels are NOT swapped.
#The 6 values in each parentheses are for the 6 DCS channels, from left to right mapping to
LSDCS0-0,LSDCS0-1,LSDCS1-0,LSDCS1-1, HSDCS0, HSDCS1
#To enable IQSWAP, change the value to 1
#example: iqswap_tx=(0 0 1 0 0 0) will enable IQ SWAP on TX of LSDCS1-0
iqswap_tx=(0 0 0 0 0 0)
iqswap_rx=(0 0 0 0 0 0)
```



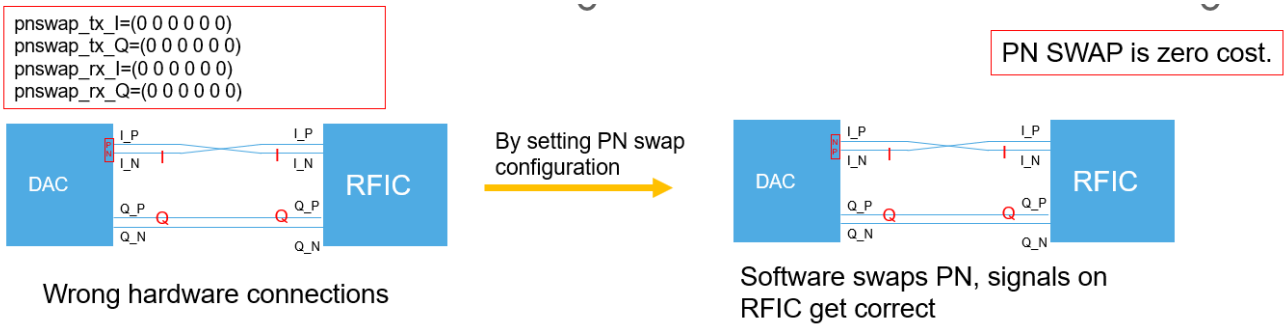
5.9 PN SWAP Test

If users made mistake in their hardware design which swaps P and N on DAC output or ADC input, they need test with PN swap. PN swap is done by VSPA in software before the signal is sent to DAC or received from ADC. The software will inverse the polarity of the signal.

To test PN SWAP at initialization stage:

- Change PN SWAP configurations in config.dat, see below red box. The default configuration is PNSWAP disabled. To enable PNSWAP, set relevant values to 1

- Reboot and start the test
- Users can use ./dump_time_domain_tx/rx.sh to check the PN SWAP result.



To test PN SWAP at runtime

To enable PN SWAP on specified antenna:

./update_qec_coeff_tx/rx.sh <ant_id> inc pnswapi

./update_qec_coeff_tx/rx.sh <ant_id> inc pnswapq

./update_qec_coeff_tx/rx.sh <ant_id> inc pnswapi pnswapq

To disable PN SWAP on specified antenna:

./update_qec_coeff_tx/rx.sh <ant_id> inc nopnswapi

./update_qec_coeff_tx/rx.sh <ant_id> inc nopnswapq

./update_qec_coeff_tx/rx.sh <ant_id> inc nopnswapi nopnswapq

5.10 SINAD Test

Test tool can do SINAD measurement, the measurement is triggered by a host command. Upon receiving the command, test tool will do the following in VSPA.

- Take 32KB (8K samples) of ant data from RX AXIQ
- Do FFT on the 8K samples (separately on I and Q)
- In the 8K samples FFT output, add up specified num of samples from specified sample as signal power, and add up the rest samples as noise power(ignore the first a few num of samples which are DC).
- Calculate SINAD values for I and Q: 10*log10(signal power/noise power)
- Write SINAD values to specified address.

Script to trigger SINAD test

./sinad_test.sh <ant_id> <start_point> <num_points> [dest_address]

- *start_point: signal start point in the 8192 FFT output, range 0-8191*
- *num_points: number of signal points from start point.*
- *dest_address: the address in host view(40bit) where SINAD result(24bytes needed) are stored, must be 4KB aligned. must be on DDR. if not specified, test tool will assign the address*
- *example: ./sinad_test.sh 4 1660 13 will test SINAD for antenna 4, signal starts from point 1660 and end at point 1672 total 13 points(400Mhz single tone at 1.9Gbps)*
- *example: ./sinad_test.sh 4 2910 13 will test SINAD for antenna 4, signal starts from point 2910 and end at point 2922 total 13 points(700Mhz single tone at 1.9Gbps).*
- *example: ./sinad_test.sh 4 3327 13 will test SINAD for antenna 4, signal starts from point 3327 and end at point 3339 total 13 points(400Mhz single tone at 983Mbps).*

The following information will be printed on console (values are just for example):

SINAD_I at address: 0x2361b09000, Q at address 0x2361b09004, data type is 32-bit float little endian.
SINAD_I: 50.234375
SINAD_Q: 50.234375

Requirements:

- Required vspa images:
  - vspa_images_LS.4T4R_100M_30K_491_245_HS.2T2R_400M_120K_1966_1966_TDDFDD_SINAD
  - vspa_images_LS.4T4R_100M_30K_245_245_HS.2T2R_400M_120K_983_983_TDDFDD_SINAD
- Required mode
  - FDD
- Required steps
  - ./boot_vspa.sh <vspa_image_folder_name>
  - ./channels_start.sh sinad
  - ./send_single_tone.sh <ant_id> <freq> <amp>
  - ./sinad_test.sh <ant_id> <start_point> <num_points> [dest_address]
- Required hardware setup
  - Test tool will enable 6T6R, users must externally loopback any one of the 6 TX to any one of the 2 HS RX.
  - Then use above steps to test SINAD on that RX.
  - Example: If HSDAC0 TX is externally looped back to HSADC1 RX, use the following steps to test 400Mhz/700Mhz at 1.9Gsps
    - ./boot_vspa.sh vspa_images_LS.4T4R_100M_30K_491_245_HS.2T2R_400M_120K_1966_1966_TDDFDD_SINAD
    - ./channels_start.sh sinad
    - ./send_single_tone.sh 4 400000000 85
    - ./sinad_test.sh 5 1660 13 0x236429e000

- ./send_single_tone.sh 4 700000000 85
- ./sinad_test.sh 5 2910 13 0x236429e000

– If software can control the external loopback from any TX to any RX, ./send_single_tone.sh and ./sinad_test.sh can be run on any ant_id repeatedly without reboot or restart.

6 PCI Bandwidth Measurement

The test tool provides a script to measure the VSPA DMA performance over PCI. As the data between host and DFE are transferred over PCI, if the PCI bandwidth is not high enough, DFE will get underrun or overrun errors thus the system will get corrupted. To know the PCI bandwidth and performance, run the following script before ./channels_start.sh.

For LA12xx, all VSPA images support PCI bandwidth measurement.

./boot_vspa.sh <vspa_image>

./measure_dma_latency.sh

For LA9310, only MEvspa_images_LS.1T1R.T4x_25M_60K_122_122_TDDFDD_LA93 supports PCI bandwidth measurement.

./boot_vspa.sh MEvspa_images_LS.1T1R.T4x_25M_60K_122_122_TDDFDD_LA93

./measure_dma_latency.sh

Measurement result:

VSPA DMA Memory read/write performance test result:

```
DDR  READ   by VSPA DMA 1 channel , 16384 bytes, cycle count   3866,   6292 ns,  20830 Mbps
DDR  READ   by VSPA DMA 2 channels, 16384 bytes, cycle count   2493,   4057 ns,  32302 Mbps
DDR  WRITE  by VSPA DMA 1 channel , 16384 bytes, cycle count   1508,   2454 ns,  53402 Mbps
DDR  WRITE  by VSPA DMA 2 channels, 16384 bytes, cycle count   1497,   2436 ns,  53794 Mbps
HRAM READ   by VSPA DMA 1 channel , 16384 bytes, cycle count    722,   1175 ns,  111538 Mbps
HRAM READ   by VSPA DMA 2 channels, 16384 bytes, cycle count    615,   1000 ns,  130944 Mbps
HRAM WRITE  by VSPA DMA 1 channel , 16384 bytes, cycle count    614,    999 ns,  131157 Mbps
HRAM WRITE  by VSPA DMA 2 channels, 16384 bytes, cycle count    555,    903 ns,  145100 Mbps
```

Above result shows VSPA can read from DDR with 20Mbps or 32Mbps with 1 or 2 DMA channels, write to DDR with 53Msps.

7 Waveform Generation by Matlab

Matlab 5G NR-TM and FRC Waveform Generation

5G NR-TM and FRC Waveform Generation - MATLAB & Simulink (mathworks.com)

Commands and Parameters

DL:

dlrefwavegen = hNRReferenceWaveformGenerator(dlnrref,bw,scs,dm,ncellid,sv)

[dlrefwaveform,dlrefwaveinfo,dlresourceinfo] = generateWaveform(dlrefwavegen);

dlrefwaveform : DL time domain samples

dlrefwaveinfo.ResourceGridBWP(or ResourceGridInCarrier): DL freq domain samples

UL:

ulrefwavegen = hNRReferenceWaveformGenerator(ulnrref,bw,scs,dm,ncellid)

[ulrefwaveform,ulrefwaveinfo,ulresourceinfo] = generateWaveform(ulrefwavegen);

ulrefwaveform : UL time domain samples

ulrefwaveinfo.ResourceGridBWP(or ResourceGridInCarrier): UL freq domain samples

Time/Freq domain samples amplitude adjust

For example,

Reducing freq domain power to avoid VSPA IFFT saturation

Increasing time domain power to full DAC range

Freq domain symbol padding

In case freq domain symbol size is not vspa dmem line(128Byte) aligned, padding zeros at symbol tail is needed

Save bin file

I followed by Q, int16, 2's complement

8 Revision history

Revision number	Release date	Description
A	30 November 2023	Initial NDA release
5.0	31 March 2025	V5.0
5.1	1 Nov 2025	V5.1

New in this release:

1. Reverse loopback support on FR1 and FR2
2. 1600Mhz bandwidth support on FR2

9 Note about the source code in the document

Example code shown in this document has the following copyright and BSD-3-Clause license:

Copyright 2023 NXP Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.
3. Neither the name of the copyright holder nor the names of its contributors may be used to endorse or promote products derived from this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

Legal information

Definitions

Draft — A draft status on a document indicates that the content is still under internal review and subject to formal approval, which may result in modifications or additions. NXP Semiconductors does not give any representations or warranties as to the accuracy or completeness of information included in a draft version of a document and shall have no liability for the consequences of use of such information.

Disclaimers

Limited warranty and liability — Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and shall have no liability for the consequences of use of such information. NXP Semiconductors takes no responsibility for the content in this document if provided by an information source outside of NXP Semiconductors.

In no event shall NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein shall be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes — NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use — NXP Semiconductors products are not designed, authorized or warranted to be suitable for use in life support, life-critical or safety-critical systems or equipment, nor in applications where failure or malfunction of an NXP Semiconductors product can reasonably be expected to result in personal injury, death or severe property or environmental damage. NXP Semiconductors and its suppliers accept no liability for inclusion and/or use of NXP Semiconductors products in such equipment or applications and therefore such inclusion and/or use is at the customer's own risk.

Applications — Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Terms and conditions of commercial sale — NXP Semiconductors products are sold subject to the general terms and conditions of commercial sale, as published at https://www.nxp.com/profile/terms, unless otherwise agreed in a valid written individual agreement. In case an individual agreement is concluded only the terms and conditions of the respective agreement shall apply. NXP Semiconductors hereby expressly objects to applying the customer's general terms and conditions with regard to the purchase of NXP Semiconductors products by customer.

Export control — This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from competent authorities.

Suitability for use in non-automotive qualified products — Unless this document expressly states that this specific NXP Semiconductors product is automotive qualified, the product is not suitable for automotive use. It is neither qualified nor tested in accordance with automotive testing or application requirements. NXP Semiconductors accepts no liability for inclusion and/or use of non-automotive qualified products in automotive equipment or applications.

In the event that customer uses the product for design-in and use in automotive applications to automotive specifications and standards, customer (a) shall use the product without NXP Semiconductors' warranty of the product for such automotive applications, use and specifications, and (b) whenever customer uses the product for automotive applications beyond NXP Semiconductors' specifications such use shall be solely at customer's own risk, and (c) customer fully indemnifies NXP Semiconductors for any liability, damages or failed product claims resulting from customer design and use of the product for automotive applications beyond NXP Semiconductors' standard warranty and NXP Semiconductors' product specifications.

Translations — A non-English (translated) version of a document, including the legal information in that document, is for reference only. The English version shall prevail in case of any discrepancy between the translated and English versions.

Security — Customer understands that all NXP products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Customer is responsible for the design and operation of its applications and products throughout their lifecycles to reduce the effect of these vulnerabilities on customer's applications and products. Customer's responsibility also extends to other open and/or proprietary technologies supported by NXP products for use in customer's applications. NXP accepts no liability for any vulnerability. Customer should regularly check security updates from NXP and follow up appropriately.

Customer shall select products with security features that best meet rules, regulations, and standards of the intended application and make the ultimate design decisions regarding its products and is solely responsible for compliance with all legal, regulatory, and security related requirements concerning its products, regardless of any information or support that may be provided by NXP.

NXP has a Product Security Incident Response Team (PSIRT) (reachable at PSIRT@nxp.com) that manages the investigation, reporting, and solution release to security vulnerabilities of NXP products.

NXP B.V. — NXP B.V. is not an operating company and it does not distribute or sell products.

Trademarks

Notice: All referenced brands, product names, service names, and trademarks are the property of their respective owners.

NXP — wordmark and logo are trademarks of NXP B.V.

Amazon Web Services, AWS, the Powered by AWS logo, and FreeRTOS — are trademarks of Amazon.com, Inc. or its affiliates.

Layerscape — is a trademark of NXP B.V.

For more information, please visit: https://www.nxp.com

Date of release: 30 November 2023
Document identifier: FR1FR2TTUG